

BẢN ĐẶC TẢ KỊCH BẢN KIỂM THỬ TỰ ĐỘNG (AUTOMATED TEST SCRIPTS)

DỰ ÁN: HỆ THỐNG GIÁM SÁT VÀ HỖ TRỢ CHĂM SÓC LINH THẢO "MỘC ĐẠO TU TIÊN"

Tài liệu này tập trung đặc tả chi tiết các kịch bản kiểm thử (Test Scripts) tự động được thiết kế và hiện thực hóa trong mã nguồn Backend. Tài liệu mô tả rõ mục tiêu, dữ liệu đầu vào, trình tự thực hiện của script và các điểm khẳng định kết quả (Assertions) nhằm xác minh tính đúng đắn của logic nghiệp vụ hệ thống.

I. TỔNG QUAN HỆ THỐNG KIỂM THỬ TỰ ĐỘNG

Bộ kịch bản kiểm thử tự động được tổ chức thành 3 nhóm kịch bản chính tương ứng với các file mã nguồn kiểm thử trong thư mục `backend/tests/`:

Tên File Script	Loại Kiểm Thử	Phân Hệ Nghiệp Vụ Mục Tiêu	Số Lượng Kịch Bản
<code>test_auth_service.py</code>	Kiểm thử đơn vị (Unit Test)	Dịch vụ xác thực, phân quyền & xử lý tài khoản Google OAuth	7 Kịch bản
<code>test_ruff_ty.py</code>	Kiểm thử tĩnh (Static Test)	Chất lượng cú pháp, định dạng và tính nhất quán kiểu dữ liệu	3 Kịch bản
<code>verify_endpoints.py</code>	Kiểm thử tích hợp (Integration Test)	Luồng nghiệp vụ liên thông API: Cấp phát, Ghép đôi, Telemetry, Dashboard, Bảng xếp hạng	13 Kịch bản

II. KỊCH BẢN KIỂM THỬ ĐƠN VỊ - DỊCH VỤ XÁC THỰC (`test_auth_service.py`)

Kịch bản này sử dụng thư viện `pytest` để kiểm thử logic nghiệp vụ cô lập của lớp `AuthService`.

1. Điều kiện tiên đề (Preconditions)

- Thiết lập đối tượng giả lập `MockAsyncSession` để thay thế kết nối PostgreSQL thật.
- Cấu hình các tham số JWT mặc định trong bộ nhớ.

2. Bảng đặc tả chi tiết các kịch bản kiểm thử đơn vị

Mã Testcase	Chức Năng Kiểm Thử	Dữ Liệu Đầu Vào (Inputs)	Trình Tự Các Bước Thực Hiện Của Script	Logic Xác Minh Kết Quả (Assertions)
TC_AUTH_01	Tạo Access Token (JWT)	<pre>user_id = UUID role = "user"</pre>	1. Gọi hàm sinh access token. 2. Dùng thư viện JWT giải mã token vừa tạo với <code>secret_key</code> hệ thống.	<pre>assert payload["sub"] == user_id assert payload["role"] == "user" assert payload["type"] == "access" assert "exp" in payload</pre>
TC_AUTH_02	Tạo Refresh Token	<pre>user_id = UUID</pre>	1. Gọi hàm sinh refresh token. 2. Giải mã và đối chiếu.	<pre>assert payload["sub"] == user_id assert payload["type"] == "refresh" assert "exp" in payload</pre>
TC_AUTH_03	Giải mã Token hợp lệ	Chuỗi JWT được ký hợp pháp bởi khóa bí mật hệ thống.	1. Gọi hàm <code>decode_token()</code> với tham số đầu vào là chuỗi JWT.	Giải mã thành công, không ném lỗi. <pre>assert decoded["sub"] == user_id assert decoded["role"] == role</pre>
TC_AUTH_04	Từ chối Token không hợp lệ	Chuỗi text ngẫu nhiên: <code>"invalid-token-string"</code>	1. Gọi hàm <code>decode_token()</code> .	Kiểm tra ngoại lệ: <pre>pytest.raises(ValueError, match="Token không hợp lệ")</pre>
TC_AUTH_05	Từ chối Token hết hạn	Token hợp lệ nhưng có trường <code>exp</code> ở 1 giờ trước.	1. Tạo thủ công JWT có <code>exp</code> đã qua. 2. Gọi hàm <code>decode_token()</code> .	Kiểm tra ngoại lệ: <pre>pytest.raises(ValueError, match="Token không hợp lệ")</pre>
TC_AUTH_06	Tạo User mới từ Google OAuth	Email: <pre>"test@moctu.com"</pre> Display Name: <pre>"Test User"</pre>	1. Thiết lập <code>MockAsyncSession</code> trả về <code>None</code> (chưa có user). 2. Gọi hàm <code>get_or_create_user()</code> .	<pre>assert user.role == "user" assert len(db.added) == 1 (đã gọi lệnh thêm vào DB) assert db.flushed is True</pre>

TC_AUTH_07	Cập nhật User đã tồn tại	Email đã tồn tại trong CSDL giả lập.	1. Đưa đối tượng User cũ vào CSDL giả lập. 2. Gọi hàm <code>get_or_create_user()</code> với tên hiển thị mới.	<code>assert user.display_name == "Test User"</code> (đã cập nhật) <code>assert len(db.added) == 0</code> (không tạo mới nhờ SQLAlchemy Unit of Work)
------------	--------------------------	--------------------------------------	--	--

3. Giải thích logic kiểm thử cốt lõi

- Cơ chế Mock Session:** Lớp `MockAsyncSession` giả lập phương thức `execute` và `add` của SQLAlchemy. Khi gọi hàm tạo người dùng, script kiểm tra danh sách `db.added` để khẳng định hệ thống có thực hiện ghi nhận bản ghi mới hay không.
- Kiểm thử biên thời gian (Edge Case):** Script tạo thử công một token có trường `exp` bị trừ đi 1 giờ so với thời điểm hiện tại để kiểm tra khả năng phát hiện và từ chối token hết hạn của hệ thống.

III. KỊCH BẢN KIỂM THỬ CHẤT LƯỢNG MÃ NGUỒN TĨNH (test_ruff_ty.py)

Kịch bản này tự động gọi các công cụ kiểm tra tĩnh ngoài hệ điều hành thông qua thư viện `subprocess` của Python nhằm đảm bảo chất lượng cấu trúc mã nguồn.

1. Điều kiện tiên đề (Preconditions)

- Môi trường máy kiểm thử đã cài đặt đầy đủ các gói: `ruff`, `ty`.

2. Bảng đặc tả chi tiết các kịch bản kiểm thử tĩnh

Mã Testcase	Chức Năng Kiểm Thử	Công Cụ Gọi Chạy	Trình Tự Các Bước Thực Hiện Của Script	Logic Xác Minh Kết Quả (Assertions)
TC_CODE_01	Ruff Linter Check	Ruff Linter	1. Script gọi lệnh hệ điều hành: <code>ruff check .</code> 2. Thu nhận mã thoát (exit code).	<code>assert result.returncode == 0</code> (Không phát hiện bất kỳ lỗi cú pháp hay vi phạm coding standard nào)
TC_CODE_02	Ruff Format Check	Ruff Formatter	1. Script gọi lệnh: <code>ruff format --check .</code> 2. Thu nhận mã thoát.	<code>assert result.returncode == 0</code> (Xác nhận toàn bộ code đã được định dạng chuẩn hóa)
TC_CODE_03	Ty Type Check	Ty Type Checker	1. Script gọi lệnh: <code>ty check .</code> 2. Thu nhận mã thoát.	<code>assert result.returncode == 0</code> (Xác nhận 100% các biến và hàm tuân thủ khai báo static type hints)

IV. KỊCH BẢN KIỂM THỬ TÍCH HỢP ENDPOINTS (`verify_endpoints.py`)

Kịch bản này là một tiến trình chạy độc lập thực hiện gọi liên tục chuỗi 13 hành động kiểm thử (Integration Scenario) trên cơ sở dữ liệu PostgreSQL thực tế để kiểm chứng luồng hoạt động tích hợp của API.

1. Quy trình cô lập dữ liệu và dọn dẹp (Database Cleanup Isolation)

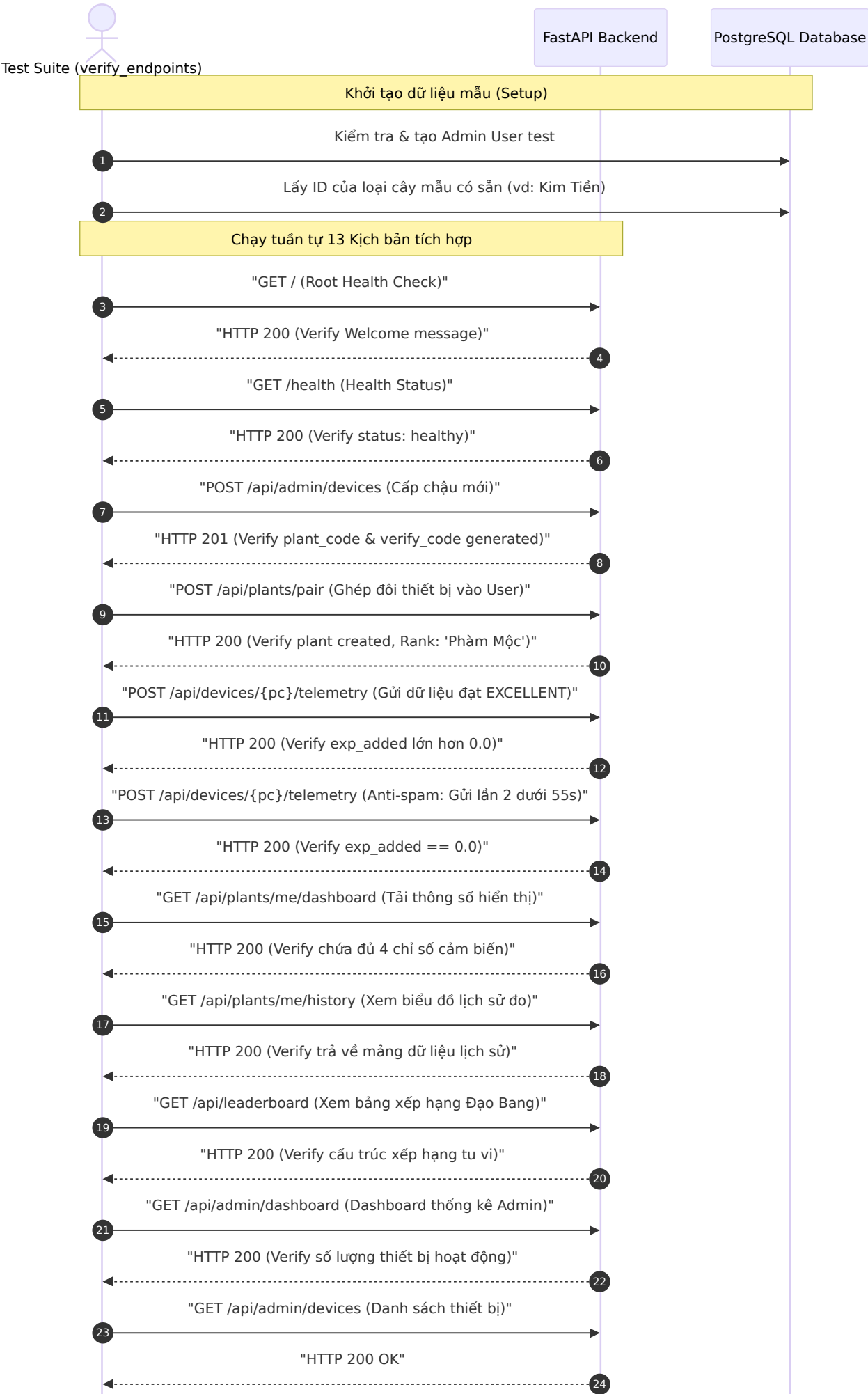
Trong mã nguồn thực tế của `verify_endpoints.py`, để dọn dẹp dữ liệu giả lập sau khi hoàn tất test, tiến trình thực thi bọc toàn bộ luồng chạy trong khối `try-finally` để đảm bảo tài nguyên và dữ liệu luôn được dọn dẹp sạch sẽ ở cuối hàm `run()`:

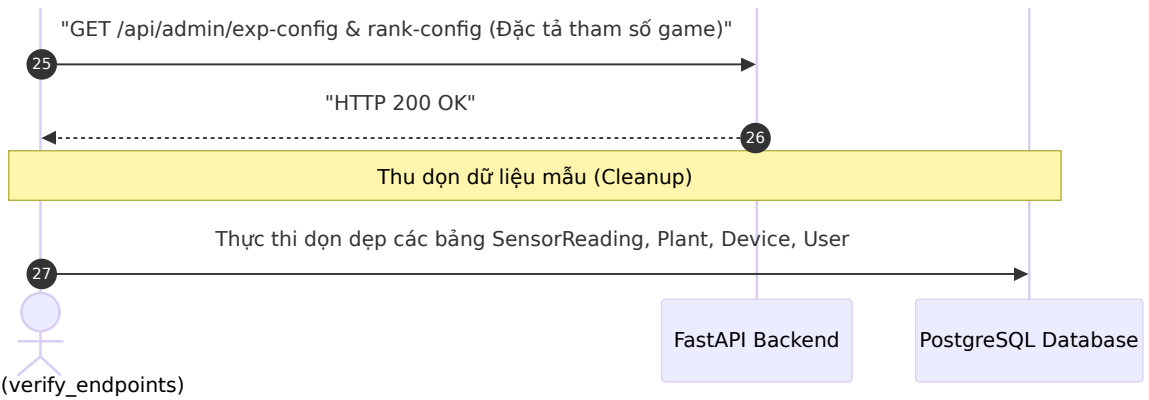
```
try:
    # [Thực hiện các bước gọi tuần tự từ Bước 1 đến Bước 13]
finally:
    # Thu dọn DB & Đóng kết nối HTTP Client
    await self.cleanup_mock_data()
    await self.client.aclose()
```

[!NOTE]

Điểm cải tiến SQA (SQA Finding - Đã hiệu chỉnh): Trước đây, mã nguồn test chưa sử dụng khối `try-finally` bao quanh các bước chạy. Nếu tiến trình bị crash giữa chừng, lệnh dọn dẹp ở cuối sẽ bị bỏ qua, dẫn đến tích tụ dữ liệu rác trong database. SQA đã khuyến nghị và đội ngũ phát triển đã hiệu chỉnh bọc toàn bộ luồng test bằng cấu trúc `try-finally`, bảo toàn tuyệt đối tính cô lập dữ liệu (Database Isolation).

2. Sơ đồ trình tự thực thi kịch bản kiểm thử tích hợp





3. Bảng đặc tả chi tiết 13 bước trong kịch bản tích hợp

Bước	Mã Testcase	Điểm Cuối API	Phương Thức	Mô Tả Kiểm Thử (Hành động của Script)	Các Khẳng Định Logic (Assertions)
1	TC_INT_01	/	GET	Health check root endpoint: Gọi API kiểm tra căn bản máy chủ.	- Mã phản hồi là HTTP 200. - Nội dung trả về chứa thông điệp chào mừng "Mộc Đạo Tu Tiên".
2	TC_INT_02	/health	GET	System health status: Kiểm tra chi tiết trạng thái hoạt động.	- Mã phản hồi là HTTP 200. - Khẳng định trường <code>status == "healthy"</code> .
3	TC_INT_03	/api/admin/devices	POST	Admin tạo mã chậu cây mới: Admin gửi yêu cầu cấp phát thiết bị chậu cây vật lý mới.	- Mã phản hồi là HTTP 201 Created. - Sinh mã <code>plant_code</code> ngẫu nhiên và băm <code>verify_code</code> bằng Bcrypt thành công.
4	TC_INT_04	/api/plants/pair	POST	Liên kết chậu cây với tài khoản: Người dùng gửi yêu cầu ghép đôi chậu cây mới cấp phát vào tài khoản của mình.	- Mã phản hồi là HTTP 200 OK. - Khởi tạo hồ sơ linh thảo ảo với Cảnh Giới mặc định là "Phạm Mộc".

5	TC_INT_05	/api/devices/{pc}/telemetry	POST	Upload dữ liệu cảm biến qua REST API, kiểm tra EXP được cộng > 0: IoT gửi gói tin cảm biến nằm trong dải lý tưởng (Độ ẩm đất = 45.0).	- Mã phản hồi là HTTP 200 OK. - Xác thực cộng EXP: <code>exp_added > 0.0</code> (Do môi trường đạt mức EXCELLENT).
6	TC_INT_06	Anti-spam Telemetry	POST	Anti-spam Telemetry: Gửi tiếp gói tin cảm biến lần thứ 2 khi thời gian cách biệt chưa đủ 55 giây kể từ lần 1.	- Mã phản hồi là HTTP 200 OK. - Xác thực chặn spam điểm: <code>exp_added == 0.0</code> (Không cộng thêm EXP).
7	TC_INT_07	/api/plants/me/dashboard	GET	Lấy dữ liệu Dashboard cây trồng, kiểm tra đủ 4 sensor keys: Người dùng yêu cầu tải dữ liệu bảng điều khiển (Dashboard).	- Mã phản hồi là HTTP 200 OK. - Dữ liệu <code>latest_readings</code> chứa đầy đủ 4 khóa cảm biến: <code>soil_moisture</code>, <code>light</code>, <code>temperature</code>, <code>humidity</code>.
8	TC_INT_08	/api/plants/me/history	GET	Lấy lịch sử cảm biến soil_moisture trong 24h: Người dùng truy vấn dữ liệu biểu đồ xu hướng cảm biến độ ẩm đất.	- Mã phản hồi là HTTP 200 OK. - Trả ra danh sách số liệu lịch sử đo đạc kèm mốc thời gian cụ thể.

9	TC_INT_09	/api/leaderboard	GET	Lấy bảng xếp hạng Tu Vi, kiểm tra trả về danh sách hợp lệ: Người dùng truy cập xem Bảng xếp hạng Đạo Bang.	- Mã phản hồi là HTTP 200 OK. - Trả về mảng danh sách xếp hạng các cây trồng trên toàn hệ thống.
10	TC_INT_10	/api/admin/dashboard	GET	Dashboard thống kê Admin: Admin yêu cầu xem dữ liệu thống kê tổng quan hệ thống.	- Mã phản hồi là HTTP 200 OK. - Trả về tổng số thiết bị đã được kích hoạt.
11	TC_INT_11	/api/admin/devices	GET	Danh sách thiết bị đã đăng ký: Admin yêu cầu xem danh sách toàn bộ thiết bị.	- Mã phản hồi là HTTP 200 OK. - Trả về danh sách chứa thiết bị được cấp phát ở Bước 3.
12	TC_INT_12	/api/admin/exp-config	GET	Lấy cấu hình hệ số EXP: Admin yêu cầu lấy danh sách hệ số EXP cấu hình.	- Mã phản hồi là HTTP 200 OK. - Trả về cấu hình hệ số EXP ứng với các mức môi trường.
13	TC_INT_13	/api/admin/rank-config	GET	Lấy cấu hình mốc Cảnh Giới: Admin yêu cầu lấy danh sách mốc Cảnh Giới.	- Mã phản hồi là HTTP 200 OK. - Trả về mốc EXP yêu cầu của các Cảnh Giới.

V. KỊCH BẢN KIỂM THỬ DỰ KIẾN TRONG TƯƠNG LAI (PLANNED TEST SUITE)

Các kịch bản dưới đây đã được thiết kế hoàn thiện về mặt giải pháp kiểm thử và dự kiến sẽ hiện thực hóa bằng mã nguồn tự động khi phân hệ Frontend và hạ tầng (Redis, Locust) được đưa vào vận hành.

1. Bảng đặc tả giải pháp kiểm thử dự kiến

Mã Testcase	Chỉ Tiêu Kiểm Thử	Môi Trường Chạy Script	Trình Tự Thực Hiện Của Script	Các Khẳng Định Logic Dự Kiến (Assertions)
-------------	-------------------	------------------------	-------------------------------	---

TC_PLAN_01	Hiển thị Dashboard realtime (UC2)	Web Browser (Playwright / Cypress)	1. Khởi chạy browser ảo. 2. Giả lập Backend emit gói dữ liệu SSE. 3. Kiểm tra thay đổi trên giao diện Dashboard.	UI tự động cập nhật dữ liệu cảm biến mới mà không reload trang. <pre>assert DOM.innerText == sensor_value</pre>
TC_PLAN_02	Biểu đồ lịch sử cảm biến (UC3)	Web Browser (Playwright)	1. Tải trang lịch sử linh thảo. 2. Cung cấp mảng dữ liệu lịch sử 7 ngày. 3. Kiểm tra canvas vẽ đồ thị Chart.js.	Đồ thị hiển thị đúng số điểm đo, vẽ dải màu ngưỡng sinh lý an toàn. <pre>assert Chart.instance.data.datasets.length > 0</pre>
TC_PLAN_03	Bảng xếp hạng Đạo Bang realtime (UC4)	Backend Benchmark Script	1. Seed 10,000 tài khoản ảo vào Redis Sorted Set. 2. Chạy lệnh truy cập bảng xếp hạng từ API.	Thời gian phản hồi API phải cực nhanh nhờ cache. <pre>assert response.latency < 10 (ms)</pre>
TC_PLAN_04	Chống XSS biệt danh (UC5)	Pytest API Client	1. Gửi request cập nhật tên cây chứa chuỗi Script độc hại: <pre><script>alert(1)</script></pre>	Hệ thống lọc sạch HTML hoặc báo lỗi ValidationError. <pre>assert "<script>" not in response.json()["name"]</pre>
TC_PLAN_05	Stress Test 500 thiết bị đồng thời	Locust Load Testing	1. Khởi tạo 500 Clients ảo gửi telemetry liên tiếp qua REST/MQTT trong 30 phút.	Đảm bảo hệ thống chịu tải tốt. <pre>assert success_rate > 99%</pre> <pre>assert response.time_avg < 100 (ms)</pre>
TC_PLAN_06	Offline Detection & Heartbeat	Integration Test Server	1. Ngắt gửi gói tin từ thiết bị vật lý. 2. Sau 5 phút, kiểm tra API trạng thái thiết bị.	Redis hết hạn key-space, chuyển trạng thái linh thảo. <pre>assert plant.status == "offline"</pre>

TC_PLAN_07	Phát hành Voucher tự động (UC2_2)	Integration Test Server	1. Gửi telemetry giúp cây tích lũy đủ EXP để "Đột phá Cảnh giới". 2. Truy vấn API ví voucher người dùng.	Kiểm tra xem mã Voucher ngẫu nhiên có được cấp phát tự động. <code>assert len(user.vouchers) == 1</code> <code>assert voucher.is_active is True</code>
------------	-----------------------------------	-------------------------	---	--

Tài liệu Đặc tả Kịch bản Kiểm thử tự động được xây dựng và cập nhật bởi nhóm phát triển hệ thống Mộc Đạo Tu Tiên.